

DocuVision: Automatic Document Scanner and Quality Analyzer

1. Cover Page

Project Title: DocuVision: Automatic Document Scanner and Quality Analyzer

Course Area: Computer Vision

Project Type: Image processing and document analysis

Submitted By: Student Name / Registration Number

Repository: GitHub repository link to be added after upload

2. Introduction

Document images captured using mobile phones often suffer from perspective distortion, poor lighting, blur, and unnecessary background regions. These problems reduce readability and make digital submission less reliable. DocuVision is a computer vision project that converts a normal camera image of a document into a clean scanned output and reports whether the image quality is acceptable.

The project applies classical computer vision concepts including grayscale conversion, filtering, edge detection, contour detection, polygon approximation, perspective transformation, adaptive thresholding, and quality feature extraction.

3. Problem Statement

The problem is to build a system that can automatically locate a document inside a captured image, correct the perspective, enhance the text region, and provide feedback about capture quality. The system should reduce manual editing and help users produce readable scanned documents from ordinary images.

4. Objectives

- * Detect the document area from an input image.
- * Correct document tilt and perspective distortion.
- * Enhance the document for improved readability.
- * Analyze brightness, contrast, sharpness, and skew.
- * Generate visual and JSON reports for evaluation.
- * Maintain modular, testable, and well-documented code.

5. Functional Requirements

ID	Functional Requirement	Description
FR1	Image Input	The system shall accept an image path from the user.
FR2	Preprocessing	The system shall resize, grayscale, blur, and edge-detect the image.
FR3	Document Detection	The system shall detect the largest quadrilateral document contour.
FR4	Perspective Correction	The system shall transform the detected document into a flat rectangular view.
FR5	Enhancement	The system shall improve readability using denoising, CLAHE, and adaptive thresholding.
FR6	Quality Analysis	The system shall calculate brightness, contrast, sharpness, and skew.

FR7 Report Generation The system shall save scanned output, visual report, and JSON quality report.

6. Non-Functional Requirements

ID	Requirement	Description
NFR1	Performance	The system should process a standard image in a few seconds on a normal laptop.
NFR2	Usability	The user should be able to run the project using one command.
NFR3	Reliability	The system should use a fallback boundary if document contour detection fails.
NFR4	Maintainability	The source code should be separated into clear modules.
NFR5	Error Handling	The system should report missing or unreadable input images.
NFR6	Resource Efficiency	The baseline should avoid large datasets and heavy deep-learning models.

7. System Architecture

The architecture contains six major components:

- * Image I/O module
- * Preprocessing module
- * Document scanner module
- * Enhancement module
- * Quality analysis module
- * Report generation module

Architecture diagram: docs/diagrams/architecture.md

8. Design Diagrams

The following diagrams are included in the repository:

- * Use Case Diagram: docs/diagrams/use_case.md
- * Workflow Diagram: docs/diagrams/workflow.md
- * Sequence Diagram: docs/diagrams/sequence.md
- * Class / Component Diagram: docs/diagrams/class_diagram.md
- * Storage Design / ER-style Diagram: docs/diagrams/storage_design.md

9. Database / Storage Design

The project does not require a relational database because it processes one image and stores generated files. The output storage consists of:

- * scanned_document.png: enhanced final scan
- * visual_report.png: four-panel pipeline visualization
- * quality_report.json: structured quality metrics and messages

This file-based design is appropriate because the project objective is image processing rather than record management.

10. Design Decisions and Rationale

Classical CV instead of deep learning: The project uses deterministic image-processing algorithms because document detection can be solved effectively without training data. This makes the project easier to run, explain, and evaluate.

Canny edge detection and contour approximation: These methods are suitable for finding page boundaries because documents usually form strong rectangular edges against a background.

Perspective transform: A four-point transform directly addresses the common problem of tilted camera images.

CLAHE and adaptive thresholding: These methods improve document readability under uneven lighting.

Quality metrics: Brightness, contrast, sharpness, and skew provide measurable feedback rather than only producing an output image.

Fallback boundary: If contour detection fails, the system still produces an output and marks that fallback was used.

11. Implementation Details

The project is implemented in Python using OpenCV and NumPy.

Main source files:

- * `src/docuvision/io_utils.py`: reads, writes, and resizes images.
- * `src/docuvision/preprocessing.py`: performs grayscale conversion, Gaussian blur, Canny edge detection, dilation, denoising, CLAHE, and adaptive thresholding.
- * `src/docuvision/scanner.py`: detects contours, orders document corners, applies perspective correction, and returns scan results.
- * `src/docuvision/quality.py`: computes brightness, contrast, sharpness, skew, and quality messages.
- * `src/docuvision/report.py`: creates the visual pipeline report and JSON report.
- * `src/docuvision/cli.py`: provides the command-line interface.

Processing pipeline:

1. Read image from disk.
2. Resize image for stable contour detection.
3. Convert image to grayscale.
4. Apply Gaussian blur.
5. Run Canny edge detection.
6. Dilate edges to connect boundaries.
7. Find contours and approximate the largest quadrilateral.
8. Apply four-point perspective transform.
9. Enhance output using denoising, CLAHE, and adaptive thresholding.
10. Compute quality metrics.
11. Save generated outputs.

12. Screenshots / Results

The repository includes a synthetic sample generator:

```
bash
python scripts/generate_sample.py
```

The project can then be executed using:

```
bash
python -m docuvision.cli --input samples/sample_document.jpg --output-dir outputs
```

Expected outputs:

- * outputs/scanned_document.png
- * outputs/visual_report.png
- * outputs/quality_report.json

The visual report shows the input with detected contour, edge map, perspective corrected image, and final enhanced output.

13. Testing Approach

Testing uses Python unittest and focuses on important computer vision logic:

- * Corner ordering correctness
- * Perspective transform output validity
- * Rectangle contour detection
- * Quality analysis on low-information images
- * End-to-end scan pipeline on a synthetic document image

Run tests:

```
bash
python -m unittest discover -s tests
```

14. Challenges Faced

- * Document images can vary significantly in lighting, background, camera angle, and blur.
- * Contour detection may fail when the document edge is weak or background contrast is low.
- * Threshold values must be balanced so the system works across multiple image conditions.
- * Skew estimation using line detection can be uncertain when the document contains few strong text lines.

15. Learnings and Key Takeaways

- * Preprocessing strongly affects the quality of later computer vision steps.
- * Canny edge detection and contour approximation are effective for structured objects such as documents.
- * Perspective transformation is a practical technique for correcting real-world camera distortion.
- * Quality evaluation makes the system more useful because it explains whether the input image is reliable.
- * Modular design makes experimentation and testing easier.

16. Future Enhancements

- * Add batch processing for multiple document images.
- * Add automatic shadow removal.
- * Add deep-learning-based document segmentation for complex backgrounds.
- * Add OCR integration for extracting text from the scanned document.
- * Add a web or desktop interface.
- * Add real mobile camera capture support.

17. References

- * OpenCV Documentation: Canny Edge Detection, Contours, Perspective Transform, Adaptive Thresholding, Hough Lines.
- * Rafael C. Gonzalez and Richard E. Woods, *Digital Image Processing*.
- * Richard Szeliski, *Computer Vision: Algorithms and Applications*.